



Universidad Alfonso X
El Sabio

Grado en Ingeniería Matemática

Gestión de datos — Proyecto final

CONSTRUCCIÓN DE UN ENTORNO ANALÍTICO Y
CÁLCULOS DE MÉTRICAS RELACIONADAS CON EL
CLIENTE

Lucía Cantos Burgos

11 de mayo de 2026

1. Planteamiento y alcance

El proyecto parte de una base de datos relacional con 17 tablas que muestran el funcionamiento de una empresa del sector de la salud: clientes, ventas, productos, almacenes, inventario, devoluciones y promociones. El objetivo principal del trabajo es transformar esa base de datos inicial, en un sistema preparado para analizar. A partir de este proceso se busca calcular el *Customer Lifetime Value* (CLTV), para poder segmentar a los clientes en grupos diferenciados que sean útiles para tomar futuras decisiones de negocio.

El trabajo se ha estructurado en varias fases; empezando restaurando la base de datos original en PostgreSQL, luego el diseño del modelo dimensional, un ETL en Python, limpiando y transformando los datos necesarios. Posteriormente se construyó una tabla analítica de clientes con cálculo de CLTV y métricas complementarias, y por último PCA + *clustering* con representación visual en Power BI. En los siguientes apartados están explicadas y justificadas las decisiones tomadas en cada fase, indicando tanto el motivo técnico como el impacto sobre el resultado final.

2. Arquitectura de datos y modelos

2.1. Restauración del origen y organización en esquemas

La base de datos se restauró en una instancia local de PostgreSQL dentro del esquema **public**, que actúa como capa *raw* (datos en bruto). Durante la restauración aparecieron algunos problemas habituales, relacionados con roles, permisos y propietarios de las tablas. Estos se resolvieron ajustando la propiedad de las relaciones para poder cargar correctamente toda la información.

Sobre esa capa *raw* se crearon dos esquemas adicionales que organizan toda la arquitectura analítica:

- **staging**: contiene las versiones limpias de las 17 tablas originales (identificadas por el sufijo `_limpia`), generadas por el ETL. Es la capa para la zona intermedia donde los datos ya han sido normalizados pero aún conservan una estructura bastante parecida a la del origen.
- **dwh**: es donde se guarda el modelo dimensional final (formado por las tablas de hechos, dimensiones y analíticas), pensado para realizar consultas y explotación.

Esta organización en tres capas permite reejecutar el ETL sin tocar el origen, validar transformaciones en **staging** antes de usarlas en el modelo final, y el esquema **dwh** está separado y preparado únicamente para el análisis, para que cualquier cambio en los datos originales no afecte directamente a los resultados finales.

2.2. Modelo entidad-relación de origen

El modelo entidad-relación del sistema original se construyó después de revisar las claves y las relaciones existentes en las 17 tablas. Para facilitar su análisis se agruparon en seis bloques según su función dentro del sistema: clientes (**customer**, **city_zone**), ventas (**sale**, **sale_item**), productos (**product**, **central_product**, **brand**, **category**), inventario (**inventory**, **central_inventory**, **warehouse**, **warehouse_location**), devoluciones (**return_item**, **return_reason**) y ofertas (**offer**, **product_offer**). El modelo ER completo se incluye como fichero adjunto (**ER_modelo.pdf**); la Figura 1 muestra una vista reducida.

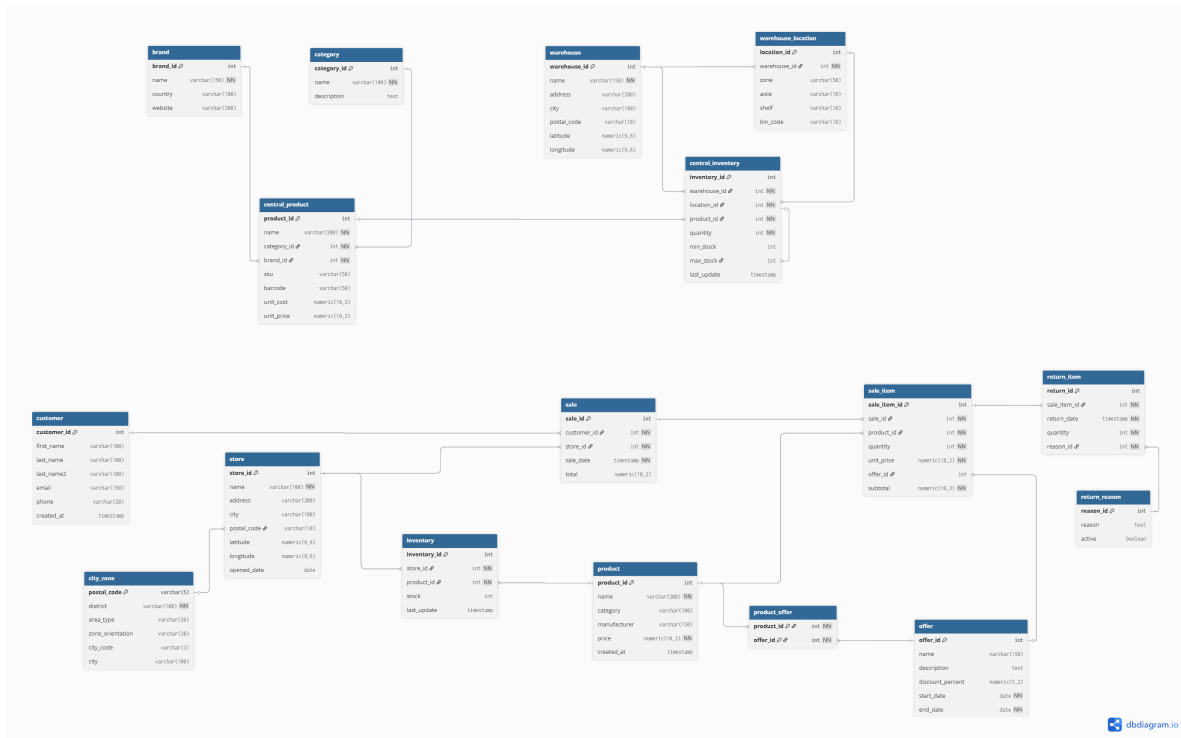


Figura 1: Modelo entidad-relación del sistema origen (17 tablas, 6 bloques funcionales).

Durante la inspección se detectó un detalle importante: la tabla **product** contiene 50 productos y **central_product** contiene 49. Esta diferencia se debe a una situación de doble fuente (sistema de tienda vs. catálogo centralizado), que se resolvió manteniendo **central_product** como referencia en el modelo dimensional, dado que aporta **unit_cost** y **unit_price** necesarios para calcular márgenes y para analizar la rentabilidad de los productos.

2.3. Modelo dimensional

A partir del modelo entidad-relación se diseñó un esquema en estrella, pensado para responder a las preguntas analíticas del enunciado. Se eligió este modelo en lugar de copo de nieve porque el volumen de dimensiones no era demasiado grande, y además, tener la información más agrupada facilita tanto las consultas SQL como el trabajo posterior de Power BI.

Las decisiones más relevantes para el modelado han sido:

- **Dos tablas de hechos diferenciadas:** **fact_sales** a nivel de línea de venta (**sale_item**) y **fact_returns** a nivel de línea de devolución. Separar las devoluciones evita duplicar registros y permite calcular tasas de devolución limpias sin riesgo de contar dos veces.
- **Dimensión temporal propia** (**dim_time**) con día, semana, mes, trimestre, año y banderas de fin de semana / periodos especiales. Esto permite todo el análisis temporal del campo **sale_date** original.
- **Marts pre-agregados:** **customer_analytics** (la primera contiene una fila por cliente con ~30 variables RFM, valor monetario, de margen y de devolución) y **mart_customer_cltv** (la segunda resume la información más importante para el dashboard). Estos marts son útiles porque dejan los datos preparados para el PCA, el clustering y la visualización; evitando que dicha herramienta tenga que recalcularlo todo.

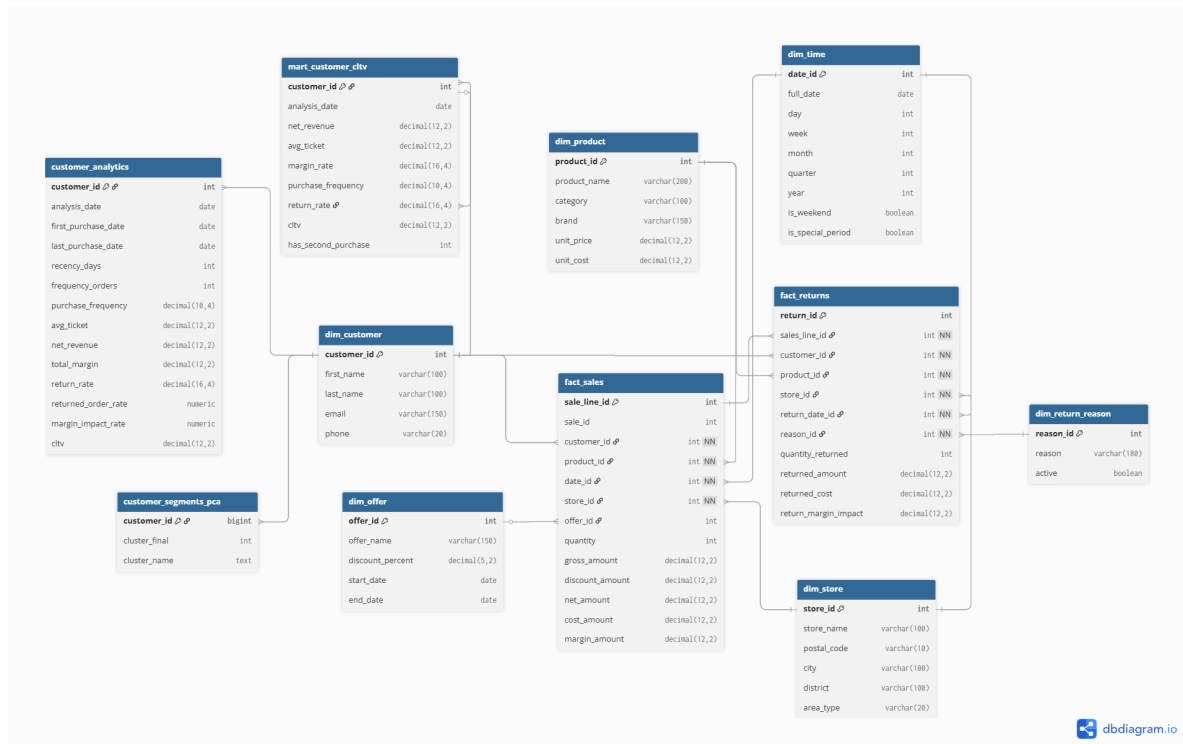


Figura 2: Modelo dimensional final: hechos **fact_sales** y **fact_returns**, dimensiones conformadas, mart analítico de cliente y tabla de segmentos del clustering.

Para completar la lectura del modelo dimensional, en la Tabla 1 se resume la función de cada una de las principales tablas finales del esquema **dw.h**. Esta tabla permite ver de forma más clara qué papel cumple cada elemento dentro del entorno analítico: dimensiones para describir el negocio, tablas de hechos para registrar ventas y devoluciones, y marts analíticos preparados para el cálculo del CLTV, el clustering y la visualización en Power BI.

Tabla	Tipo	Granularidad	Uso principal
<code>dim_customer</code>	Dimensión	1 fila por cliente	Información descriptiva del cliente y enlace con la analítica individual.
<code>dim_product</code>	Dimensión	1 fila por producto	Análisis por producto, marca, categoría, precio y coste.
<code>dim_store</code>	Dimensión	1 fila por tienda	Análisis comercial y geográfico de la actividad.
<code>dim_time</code>	Dimensión	1 fila por fecha	Análisis temporal por día, semana, mes, trimestre, año y periodos especiales.
<code>dim_offer</code>	Dimensión	1 fila por oferta	Identificación de promociones y descuentos aplicables.
<code>dim_return_reason</code>	Dimensión	1 fila por motivo	Clasificación de los motivos de devolución.
<code>fact_sales</code>	Hecho	1 fila por línea de venta	Cálculo de ingresos, unidades vendidas, ticket y margen.
<code>fact_returns</code>	Hecho	1 fila por línea devuelta	Cálculo de devoluciones, motivos e impacto económico.
<code>customer_analytics</code>	Mart analítico	1 fila por cliente	Base del CLTV, métricas RFM, rentabilidad, frecuencia y riesgo de devolución.
<code>mart_customer_cltv</code>	Mart de visualización	1 fila por cliente	Tabla preparada para el dashboard de CLTV y análisis comercial.
<code>customer_segments_pca</code>	Resultado analítico	1 fila por cliente	Segmento final, componentes PCA y cluster asignado.

Cuadro 1: Resumen de las principales tablas finales del esquema `dwh`, indicando su tipo, granularidad y uso dentro del entorno analítico.

3. Proceso ETL

El ETL se ha desarrollado en Python (`ETL.py`) con `pandas` para transformación y `SQLAlchemy` para la conexión con PostgreSQL. La elección de Python frente a SQL es porque era necesario aplicar reglas de limpieza más flexibles, generar variables nuevas y producir al mismo tiempo tres formatos de salida (CSV, Excel y tablas `staging`).

Extracción. El proceso revisa todas las tablas del esquema `public` usando la información de `information_schema.tables` y carga cada una en un diccionario de `DataFrame`. Trabajar con un único contenedor en memoria simplifica enormemente las transformaciones posteriores, ya que permite acceder a cada tabla de forma ordenada y también simplifica la generación de informes de calidad.

Transformación. Las reglas aplicadas son las mismas para todas las tablas (normalización general); con algunas reglas específicas para los problemas detectados durante la revisión inicial:

- Normalización de los nombres de las columnas a minúsculas y eliminación de espacios innecesarios.
- Limpieza de columnas de texto: aplicando `strip` y sustituyendo cadenas vacías o valores de nulos ("`nan`", "`None`", "`NULL`") por `pd.NA`.
- Conversión automática a formato de fecha: `datetime` de toda columna cuyo nombre contenga `date`, `_at` o `last_update`, con `errors=coerce` para no perder filas por formatos incorrectos.
- Imputación específica en `brand` y `city_zone`: los valores nulos se sustituyen por "`Desconocido`" en lugar de eliminarse, ya que la tabla `customer` y `product` dependen de claves foráneas relacionadas con ellas y borrar registros podría romper las relaciones entre tablas.

- En `sale_item` se crea la variable booleana `tiene_oferta` a partir de `offer_id`, y los nulos de `offer_id` se convierten en 0 (“venta sin oferta”). Esto permite filtrar y agregar los datos más fácilmente sin tener que tratar los NULL en cada consulta posterior.
- Comprobación de duplicados (no se detectaron registros duplicados en ninguna de las 17 tablas, por lo que no fue necesario eliminar ninguna fila, pero se implementó el código para hacerlo si es necesario).

Carga. El ETL genera tres salidas en paralelo: 17 archivos CSV individuales (uno por cada tabla limpia), un Excel consolidado con una hoja por tabla, y la carga de las tablas en el esquema `staging` de PostgreSQL mediante `to_sql` con `if_exists=replace`. Esto responde a funciones diferentes: los CSV para una revisión rápida, el Excel para revisión manual, y `staging` para alimentar el modelo dimensional.

Observación sobre las ofertas. La tabla `offer` contiene una única promoción activa, y `product_offer` la relaciona con 6 productos. Esta fue la razón por la que más adelante no incluimos las variables de descuento en el PCA: no aportarían variabilidad real entre clientes, lo que no ayudaría a diferenciarlos.

4. Tabla analítica y cálculo del CLTV

El `mart_dwh.customer_analytics` es la tabla central del análisis. Contiene 5.750 filas (una por cliente) y reúne toda la información necesaria para evaluar el valor, comportamiento, rentabilidad y riesgo de cada cliente. Así no es necesario volver a calcular las mismas agregaciones cada vez que se quiera hacer una consulta o hacer una visualización.

Variables derivadas más relevantes: fecha de primera, segunda y última compra; recencia, recogida en (`recency_days`); duración de la relación con el cliente (`customer_lifetime_days/years`); número de pedidos (`frequency_orders`); frecuencia de compra anual (`purchase_frequency`); ticket medio (`avg_ticket`); revenue bruto y neto; margen total y `margin_rate`; volumen de devoluciones, `return_rate`, `returned_order_rate` y `margin_impact_rate` (proporción del margen anulada por devoluciones).

Fórmula de CLTV. Siguiendo la fórmula planteada en el enunciado, se calculó:

$$\text{CLTV} = \text{Ingresos}_t \times \text{Margen}_t \times \text{Frecuencia}_t \times R_t$$

donde Ingresos_t es el revenue neto del cliente (ventas - importe devuelto), Margen_t es el `margin_rate` medio, Frecuencia_t es `purchase_frequency` (pedidos/año) y R_t es `customer_lifetime_years` (años de relación, mínimo 1 para evitar borrar clientes nuevos). Esta forma permite que cualquier componente cercana a cero anule el CLTV del cliente, lo que tiene sentido de negocio: un cliente con margen negativo o sin actividad reciente tiene baja puntuación aunque haya gastado dinero en algún momento.

La distribución del CLTV resultante es muy asimétrica: la mediana está muy por debajo de la media, y unos pocos clientes premium concentran gran parte del valor total. Esta asimetría es la razón por la que el PCA posterior requiere winsorización y transformación logarítmica.

5. Métricas complementarias

Además del CLTV, el enunciado permite añadir 2 métricas adicionales. He elegido unas que pienso que complementan bien, porque ayudan a ver aspectos que no se muestran en su totalidad.

1. Tasa de devolución e impacto en margen. El CLTV sirve para medir volumen esperado de un cliente, pero no refleja la calidad de la relación. Un cliente puede tener CLTV elevado porque compra mucho, pero se devuelve gran cantidad de productos, puede acabar no siendo rentable para la

empresa. Por eso se calculan `return_rate` (proporción de líneas devueltas), `returned_order_rate` (proporción de pedidos con al menos una devolución) y `margin_impact_rate` (porcentaje del margen bruto anulado por las devoluciones). La tasa media de devolución por volumen se sitúa en torno al 6 %, mientras que la tasa económica mostrada en el dashboard se calcula sobre el importe devuelto respecto a los ingresos. Esto esconde clientes con tasas extremas que el clustering identifica como segmentos diferenciados.

2. Rentabilidad neta del cliente. Esta métrica se calcula como el margen bruto menos el coste asociado a las devoluciones, guardado en (`return_margin_impact`). Es la métrica que combina las dos anteriores en un único indicador económico: cuánto dinero, en términos de margen, deja el cliente tras descontar sus devoluciones. Esta variable es la que mejor separa segmentos rentables de segmentos más problemáticos.

6. Análisis de Componentes Principales

6.1. Selección y preparación de variables

Para aplicar el PCA se seleccionaron 9 variables de la tabla analítica de clientes, que recogen las tres dimensiones de interés (valor, comportamiento y riesgo): `cltv`, `recency_days`, `frequency_orders`, `purchase_frequency`, `customer_lifetime_days`, `avg_ticket`, `net_revenue`, `return_rate` y `returned_order_rate`.

Se excluyeron:

- Las variables de descuento, por la baja presencia de ofertas en los datos (una sola promoción activa).
- `total_margin` y `margin_rate`, porque su correlación con `CLTV` y `net_revenue` es prácticamente lineal y solo introducirían redundancia en PC1.
- `margin_impact_rate`, que se reserva para perfilar los clusters una vez creados, pero no entra en el PCA por su concentración en cero (la mayoría de clientes no tienen devoluciones).

6.2. Tratamiento previo

Antes de aplicar el PCA, ha sido necesario preparar las variables para que los valores extremos o las grandes diferencias de escala, no afecten al resultado:

1. Sustitución de $\pm\infty$ por `NaN` y de `NaN` por 0 (los nulos en métricas agregadas significan “cliente sin actividad de ese tipo”).
2. Winsorización al percentil 1 y 99 para acotar los *outliers* extremos sin perder masa de datos.
3. Transformación logarítmica (`np.log1p`) en las variables monetarias y de frecuencia, donde la asimetría es severa.
4. Estandarización con `StandardScaler` para que el PCA no quede dominado por las variables de mayor escala.

6.3. Resultados e interpretación

Las tres primeras componentes explican el 94,92 % de la varianza acumulada, con un reparto de PC1: 59,52 %, PC2: 26,33 % y PC3: 9,07 %. El umbral del 80 % se alcanza ya con dos componentes (85,85 %) y el del 90 % se cruza al añadir la tercera. Se decidió conservar tres en lugar de las dos primeras porque PC3, aunque explique menos varianza individual, aporta una dimensión sobre cuánto tiempo ha pasado desde su última compra, esto resulta clave para distinguir clientes dormidos de clientes recientes en el *clustering* posterior.

La interpretación de cada componente se obtiene de los *loadings*, es decir, observando las variables con más peso en cada componente:

- **PC1 — Valor y recurrencia:** asociada principalmente a variables como `cltv`, `net_revenue`, `frequency_orders`, `purchase_frequency`, `customer_lifetime_days` y `avg_ticket`. Resume el valor económico del cliente y su nivel de recurrencia.
- **PC2 — Riesgo de devolución:** esta componente está dominada por `return_rate` y `returned_order_rate`. Se interpreta como una dimensión relacionada con el riesgo que suponen las devoluciones de cada cliente.
- **PC3 — Inactividad:** prácticamente explicada por `recency_days`. Identifica clientes que llevan mucho sin comprar y permite separarlos de los clientes activos.

En conjunto, esta estructura es estable, interpretable, fácil de interpretar, y lleva a tres preguntas de negocio: ¿cuánto vale el cliente?, ¿qué riesgo introduce?, ¿sigue activo?; por lo que se usa como base del clustering y para construir segmentos de clientes con sentido empresarial.

7. Clustering K-Means y segmentación final

7.1. Selección del número de clusters

Se evaluaron soluciones de $k = 3$ a $k = 8$ sobre las tres componentes principales, midiendo simultáneamente inercia, *silhouette score*, índice Davies-Bouldin e índice Calinski-Harabasz. La métrica óptima por *silhouette* correspondía a $k = 3$, pero al comprobar la distribución mostraba que esa solución concentraba más del 80 % de los clientes en un único cluster. Aunque matemáticamente esta solución podía parecer buena, desde un punto de vista comercial no es demasiado útil, ya que deja a la mayoría de clientes dentro de un grupo poco diferenciado y bastante grande.

Las soluciones con $k = 5$ y $k = 6$ se compararon más en detalle, analizando sus segmentos con las medias y medianas de todas las variables analíticas. Finalmente se eligió $k = 6$: la solución de 5 mantenía un único bloque demasiado heterogéneo de “clientes inactivos”, mientras que la de 6 lo dividía en dormidos de ticket medio y dormidos de bajo ticket, lo que habilita estrategias comerciales diferenciadas.

El modelo final obtiene un *silhouette score* de 0,515 y el mayor Calinski-Harabasz entre las soluciones evaluadas. Estos resultados indican una calidad razonable para un problema realista de segmentación de clientes, especialmente teniendo en cuenta que las variables de comportamiento suelen ser desiguales y asimétricas.

7.2. Perfilado de los seis segmentos

ID	Segmento	N	%	CLTV	Recencia	Rasgos clave
1	Premium recurrentes	750	13,04	4.601,90	—	Freq. 20 pedidos, revenue 11.504€, return rate 2,5 %
3	Recientes bajo valor	1.359	23,63	67,28	108 d	1 pedido, ticket 168€, sin devoluciones
2	Dormidos ticket medio	1.742	30,30	102,04	1.119 d	1 pedido, ticket 255€, sin devoluciones
5	Dormidos bajo ticket	1.459	25,37	23,71	1.047 d	1 pedido, ticket 59€, sin devoluciones
4	Alta devolución parcial	141	2,45	53,02	780 d	Return rate 47,9 %, impacto margen 94,3 %
0	Devolución total	299	5,20	0,00	814 d	Return rate 100 %, valor neto anulado

Cuadro 2: Perfilado de los seis segmentos sobre los 5.750 clientes. Los segmentos 1 y 3 son los prioritarios desde el punto de vista comercial (fidelización y activación, respectivamente); los segmentos 0 y 4 requieren análisis de causa raíz sobre los motivos de devolución.

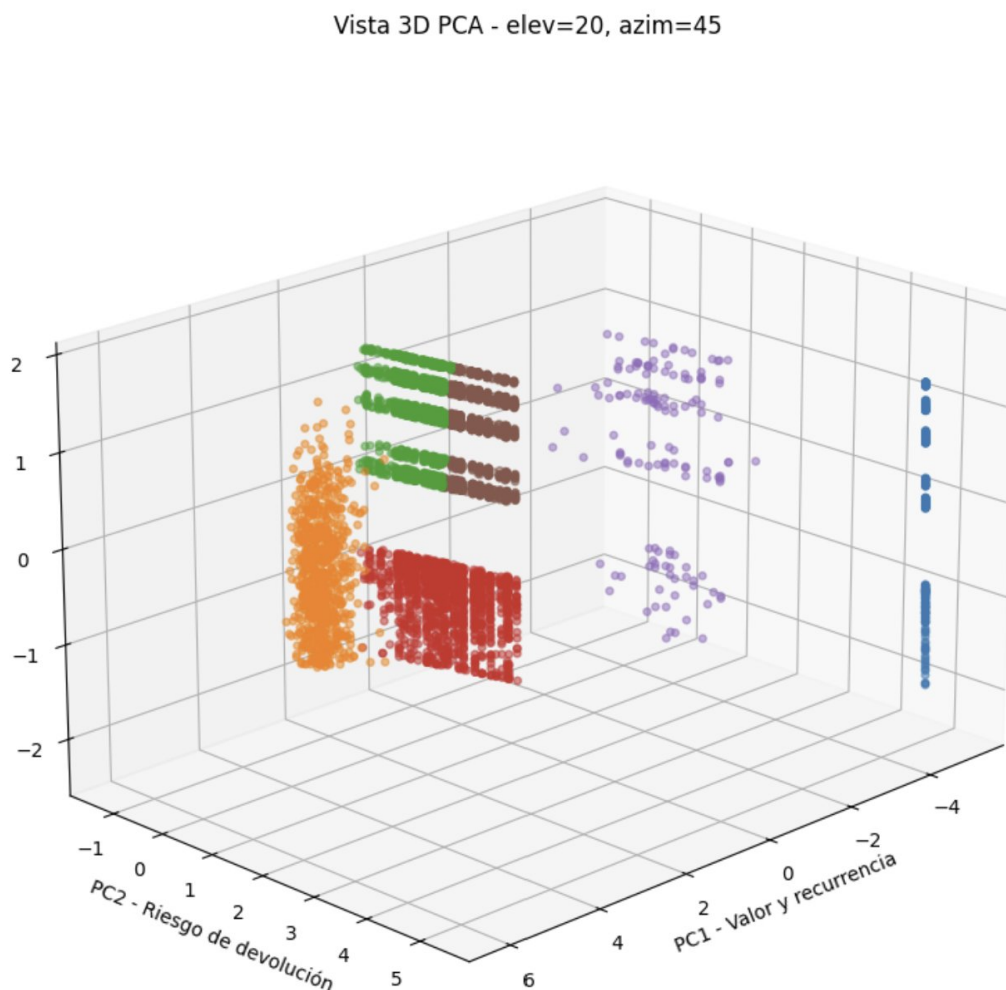


Figura 3: Proyección de los 5.750 clientes en el espacio PCA tridimensional, coloreados por segmento final. PC1 separa premium (extremo derecho) del resto; PC2 aísla los clientes con devoluciones; PC3 distingue recientes de dormidos.

La asignación final se guardó como `dwh.customer_segments_pca`, lo que permite cruzarla con cualquier dimensión del DWH desde Power BI con un JOIN directo por `customer_id`.

8. Explotación visual: dashboard Power BI

El dashboard final se conecta directamente al modelo dimensional guardado en el esquema `dwh` y se organiza en una estructura visual de arriba abajo: tarjetas con los KPI generales (número de clientes, CLTV medio, ingresos, tasa de devolución), seguidas de visualizaciones por segmento (distribución de clientes, CLTV agregado por cluster, comparativa de tasas de devolución) y, por último, detalle por producto y zona geográfica.

Tratamiento del CLTV en la visualización. La fuerte asimetría del CLTV hace que un gráfico de dispersión cliente a cliente sea ilegible: unos pocos premium aplastan visualmente al resto. Para resolverlo se crearon *buckets* de CLTV (bajo, medio, alto) usando los cuartiles, lo que convierte una variable continua en categorías comparables y permite leer la distribución de un vistazo. Los importes monetarios se formatean con símbolo de euro al inicio (formato monetario estándar de Power BI) y las tasas como porcentajes.

9. Validaciones y limitaciones

Después de cada fase del pipeline se aplicaron controles de calidad: número de filas y columnas por tabla, ausencia de duplicados en claves primarias, coherencia de claves foráneas (todo `customer_id` en `sale` existe en `customer`; todo `sale_item_id` en `return_item` existe en `sale_item`), validación del recuento de 5.750 clientes en `customer_analytics` y revisión de las distribuciones de las métricas antes y después del tratamiento previo al PCA. Todo para comprobar que los datos seguían siendo coherentes.

Control aplicado	Resultado esperado	Resultado obtenido
Tablas originales restauradas en <code>public</code>	17	17
Tablas limpias cargadas en <code>staging</code>	17	17
Clientes en <code>customer_analytics</code>	5.750	5.750
Clientes segmentados en <code>customer_segments_pca</code>	5.750	5.750
Duplicados detectados en tablas limpias	0	0
Coherencia <code>sale.customer_id</code> → <code>customer.customer_id</code>	Correcta	Correcta
Coherencia <code>return_item.sale_item_id</code> → <code>sale_item.sale_item_id</code>	Correcta	Correcta
Generación de salidas del ETL	CSV, Excel y PostgreSQL	CSV, Excel y PostgreSQL

Cuadro 3: Resumen de los principales controles de calidad aplicados durante el pipeline, desde la restauración inicial hasta la generación de las tablas analíticas finales.

Las principales limitaciones son:

- La parte promocional es muy débil (una sola oferta activa), por lo que la sensibilidad al precio no ha podido analizarse; no se sabe cómo afectan los descuentos o promociones al comportamiento de los clientes.
- El cálculo de R_t utiliza la ventana histórica disponible como aproximación de vida útil del cliente; en un escenario real convendría calibrar con un modelo probabilístico (BG/NBD o Pareto/NBD) que estime la probabilidad de que el cliente siga activo.
- El segmento 0 (“devolución total”) debería revisarse con detalle sobre los motivos de devolución antes de aplicar políticas comerciales, porque el patrón “CLTV cero por devolución completa” podría también responder a anomalías de registro o estar causado por errores.

10. Conclusiones

La decisión técnica más importante fue mantener tres componentes principales y trabajar con $k = 6$, a pesar de que la métrica favorecía $k = 3$; esta decisión resume el criterio seguido durante el trabajo: no elegir siempre el mejor resultado matemático, sino la que tenía más sentido desde un punto de vista de negocio e interpretabilidad de los clientes.

El resultado final es una segmentación de los 5.750 clientes en seis perfiles operables: un núcleo del 13 % (premium recurrentes) que concentra el valor, dos grupos con potencial (recientes y dormidos de ticket medio) sobre las que orientar la inversión comercial, un grupo de baja prioridad (dormidos de bajo ticket) y dos segmentos de riesgo (devolución total y alta devolución parcial) que requieren análisis específico antes de cualquier acción promocional.

Esta clasificación queda guardada en DWH, para poder reutilizarse con más datos, para nuevos análisis sin necesidad de reconstruir todo el pipeline desde cero.